PeerLearn: Software Architecture & Design Specification (SAD)

PeerLearn SAD v1.1 describes how the system is built: a three-layer web application with 12 components, a REST API of 14 groups of endpoints, and security built into each layer. It follows the IEEE 1016 design description outline and traces every part back to the SRS. Team: Amogh Shetty (PES1UG24AM034), Aarush Patil (PES1UG24AM005), Abheek Sen (PES1UG24AM009), Allen Anish (PES1UG24AM029).

Revision history. v1.0: version submitted for Part 1. v1.1 (31 Oct 2026): team members added; data design adds the email_outbox and revoked_tokens tables; API design lists the supporting endpoints; error handling adds ACCOUNT_SUSPENDED; new section 3.6 Deployment and DevOps.

1. Introduction

1.1 Purpose. To show the structure of PeerLearn and the design decisions that make it meet the SRS, so the team can build it in two sprints without disagreeing about how the parts fit.

1.2 Scope. The document covers the architecture (pattern, components, security) and the design (database, sequence diagrams, API, error handling) for all requirements in SRS v1.1.

1.3 References. PeerLearn SRS v1.1; PeerLearn Test Plan v1.1; IEEE 1016 (software design descriptions).

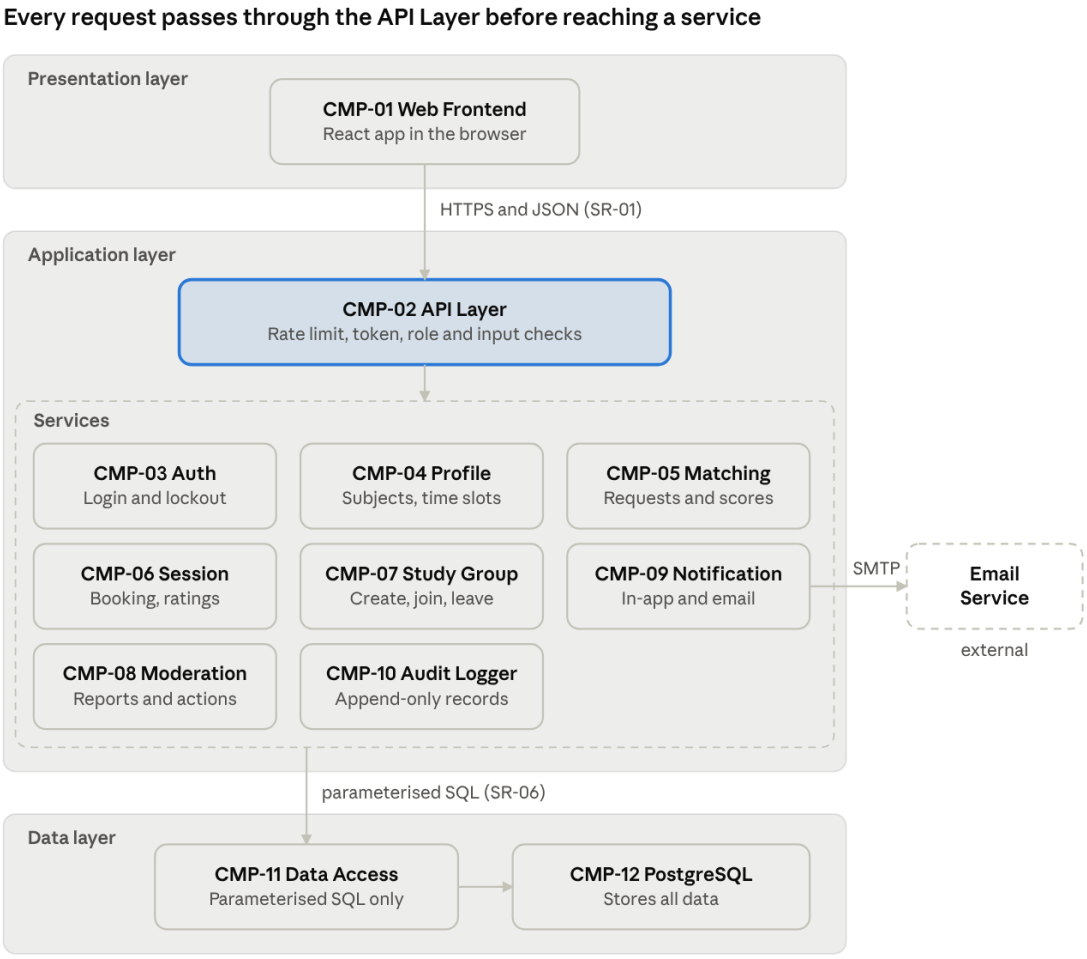
1.4 Design goals. In priority order: security of user data (SO-1 to SO-4), simplicity so four students can finish in two weeks, testability of each module, and response times of NFR-02 and NFR-03.

2. Architecture

2.1 Architectural pattern. PeerLearn uses a **layered (three-tier) client-server architecture**: a presentation layer (browser frontend), an application layer (REST API and services) and a data layer (database). A layer may only call the layer directly below it. This keeps the code easy to split between four people, lets each layer be tested alone and puts security checks in one place.

Option	Decision	Reason
Layered three-tier monolith	Chosen	Simple to build and deploy in two sprints; enough for 1,000 users (NFR-03)
Microservices	Rejected	Too much set-up and network complexity for a one-week sprint
Serverless functions	Rejected	Harder to test locally and to hold a shared PostgreSQL connection

2.2 Component diagram and description. Figure 1 shows the components and which one calls which. The table explains what each one does.



component diagram · 3 layers, 12 components, 1 external service

Figure 1: Component diagram of PeerLearn. The browser talks only to the API Layer; the API Layer calls the services; only the Data Access Layer touches the database.

ID	Component	Responsibility	Main requirements
CMP-01	Web Frontend	React single-page app with screens for each role; sends HTTPS requests to the API	FR-13, NFR-05
CMP-02	API Layer	Express routes plus middleware that applies rate limiting, token check, role check and input validation before any controller runs	NFR-04, SR-04, SR-05, SR-06
CMP-03	Auth Service	Registration, login, bcrypt hashing, token issue,	FR-01, FR-02, NFR-01, SR-

ID	Component	Responsibility	Main requirements
		account lockout	08
CMP-04	Profile Service	Tutor subjects and weekly availability	FR-03, FR-04
CMP-05	Matching Service	Stores help requests; calculates and ranks match scores	FR-05, FR-06, FR-07, FR-15, NFR-03
CMP-06	Session Service	Booking, accept or decline, 48-hour expiry, ratings, contact reveal	FR-08, FR-09, FR-14, SR-03
CMP-07	Study Group Service	Create, join and leave groups; enforce size limits	FR-10
CMP-08	Moderation Service	Reports and Admin actions	FR-12
CMP-09	Notification Service	Creates in-app notifications and passes emails to the email service	FR-11
CMP-10	Audit Logger	Writes append-only records of logins, bookings and Admin actions	NFR-06, SR-07
CMP-11	Data Access Layer	Parameterised queries; the only code that talks to the database	SR-06
CMP-12	PostgreSQL Database	Stores all application data	All

The external **Email Service** (SMTP) is used only by CMP-09.

2.3 Traceability to requirements. Every SRS requirement is implemented by at least one component and checked by at least one test case.

Requirement	Component	Test case
FR-01, FR-02	CMP-03	TC-01, TC-02, TC-03, TC-04
FR-03, FR-04	CMP-04	TC-05
FR-05, FR-06, FR-07, FR-15	CMP-05	TC-06
FR-08, FR-14	CMP-06	TC-07, TC-08
FR-09	CMP-06	TC-09
FR-10	CMP-07	TC-10

Requirement	Component	Test case
FR-11	CMP-09	TC-07, TC-08
FR-12	CMP-08	TC-11
FR-13, NFR-05	CMP-01	TC-17
NFR-01, SR-02	CMP-03, CMP-02	TC-12
NFR-02	CMP-02, CMP-11	TC-18
NFR-03	CMP-05	TC-13
NFR-04, SR-08	CMP-02, CMP-03	TC-14
NFR-06, SR-07	CMP-10	TC-16
SR-01, SR-04	Reverse proxy, CMP-02	TC-19
SR-03	CMP-06	TC-07
SR-05	CMP-02	TC-04
SR-06	CMP-02, CMP-11	TC-15

2.4 Security architecture. Security is applied at the boundaries between browser and API, and between API and database.

Layer	Security mechanism	Requirement
Network	A reverse proxy ends HTTPS (TLS 1.2 or higher) and redirects plain HTTP	SR-01
API Layer	Middleware runs in this fixed order: rate limiter, token check, role check, input validation, then the controller	NFR-04, SR-04, SR-05, SR-06
Auth Service	bcrypt hashing; 2-hour tokens; lock after 5 failed logins	NFR-01, SR-04, SR-08
Application	Response serialisers remove password hashes; contact details returned only after acceptance	SR-02, SR-03
Data Access Layer	Parameterised queries only; the application database user cannot update or delete audit records	SR-06, SR-07
Configuration	Secrets such as the token key and database password come from environment variables, never from the repository	SO-1

The **trust boundaries** are the browser (untrusted) to the API, and the API to the database. Everything that crosses the first boundary is validated.

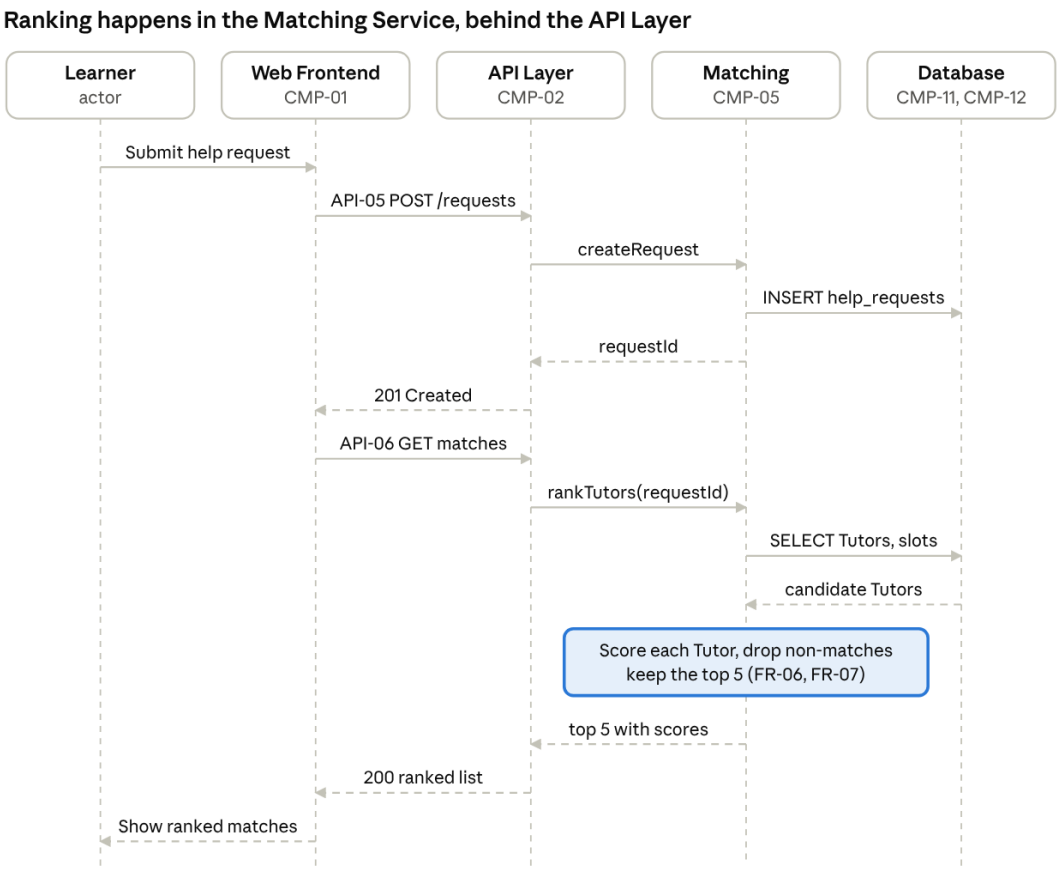
3. Design

3.1 Data design. The database has these main tables. Each foreign key links a row to its owner so that deleting a user removes their private data.

Table	Key fields	Used by
users	id, name, email (unique, @pes.edu), password_hash, role, phone, status, failed_logins, locked_until	CMP-03
tutor_subjects	id, tutor_id, subject, proficiency	CMP-04
availability_slots	id, tutor_id, weekday, start_time, end_time	CMP-04
help_requests	id, learner_id, subject, topic, preferred_time, urgency	CMP-05
sessions	id, request_id, learner_id, tutor_id, slot_time, status (pending, accepted, declined, expired, completed), created_at	CMP-06
ratings	id, session_id (unique), score 1 to 5, comment	CMP-06
study_groups, group_members	id, subject, max_size, owner_id; group_id, user_id	CMP-07
reports	id, reporter_id, reported_id, reason, status	CMP-08
notifications	id, user_id, message, read, created_at	CMP-09
audit_log	id, user_id, action, timestamp	CMP-10
email_outbox	id, to_email, subject, body, attempts, next_attempt_at, sent_at	CMP-09
revoked_tokens	jti, expires_at (tokens ended by logout)	CMP-03

3.2 UML sequence diagrams. Two key scenarios are drawn: SD-1 shows how a help request turns into ranked matches (FR-05, FR-06, FR-07, FR-15) and SD-2 shows how a session is requested and accepted (FR-08, FR-11, FR-14).

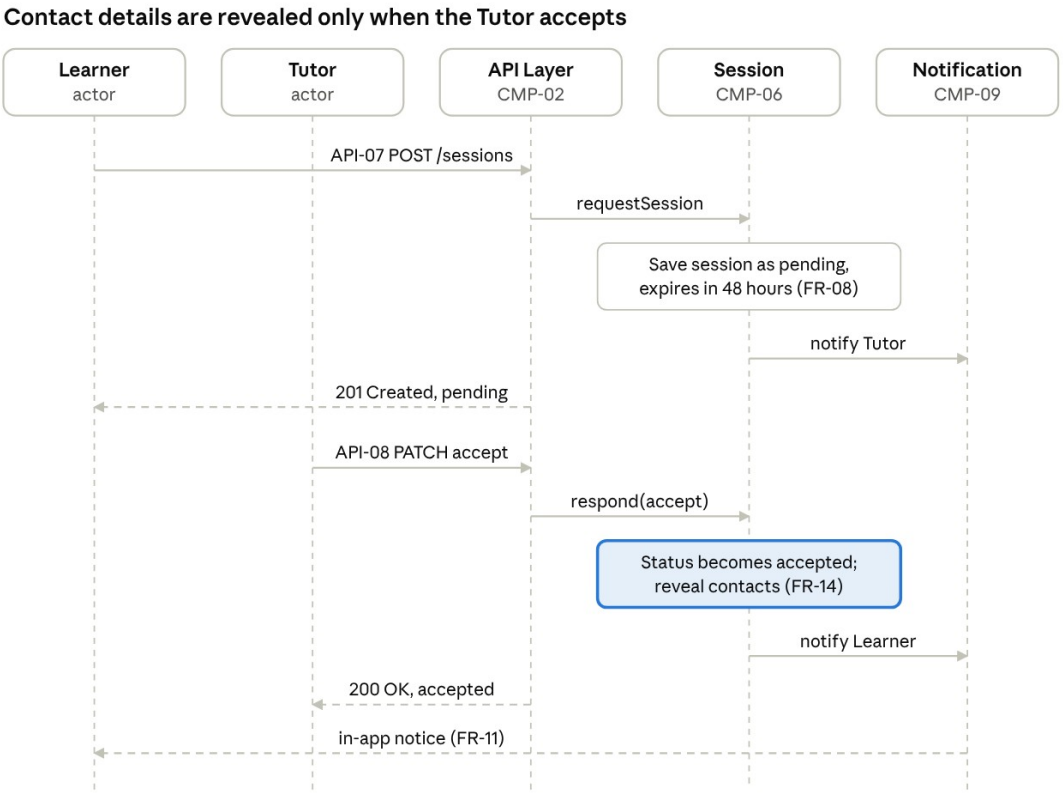
Figure 2: SD-1, Create a help request and view ranked matches.



sequence diagram SD-1 · 5 participants, 13 messages

The request is saved first (API-05); the ranking in the second call (API-06) is done by the Matching Service from data read in the database. Dashed arrows are replies.

Figure 3: SD-2, Request a session and accept it.



sequence diagram SD-2 · 5 participants, 9 messages

The Learner's request stays pending until the Tutor acts (API-08). Only then does the Session Service make email and phone visible, and both users get an in-app notification. Dashed arrows are replies.

3.3 API design. The API is REST over HTTPS using JSON. All endpoints except register and login need an `Authorization: Bearer <token>` header (SR-04). Paths start with `/api`.

ID	Method and path	Role	Purpose	Requirements
API-01	POST /auth/register	Public	Create an account; 201, or 400 or 409 on error	FR-01
API-02	POST /auth/login	Public	Return a token; 200, or 401 or 423 on error	FR-02, SR-08
API-03	POST /tutors/me/subjects ; PUT and DELETE /tutors/me/subjects /{id}	Tutor	Manage subjects	FR-03
API-04	POST /tutors/me/availabil	Tutor	Add a slot; 409 if it	FR-04

ID	Method and path	Role	Purpose	Requirements
	ity		overlaps	
API-05	POST /requests	Learner	Create a help request	FR-05
API-06	GET /requests/{id}/matches	Learner	Return the top 5 Tutors with scores	FR-06, FR-07, FR-15
API-07	POST /sessions	Learner	Request a slot with a Tutor	FR-08
API-08	PATCH /sessions/{id}	Tutor	Accept or decline a request	FR-08, FR-14
API-09	POST /sessions/{id}/rating	Learner	Rate a completed session	FR-09
API-10	POST /groups; POST and DELETE /groups/{id}/join	Learner, Tutor	Create, join and leave a group	FR-10
API-11	GET /notifications	Learner, Tutor	List notifications	FR-11
API-12	POST /reports	Learner, Tutor	Report a user	FR-12
API-13	PATCH /admin/reports/{id}	Admin	Dismiss, warn or suspend	FR-12
API-14	GET /dashboard	Learner, Tutor	Upcoming sessions, pending requests, rating	FR-13

Supporting endpoints. The screens also use POST /auth/logout, GET and PUT /users/me, GET /users/{id}, GET /tutors/me, DELETE /tutors/me/availability/{id}, GET /requests, GET /sessions, GET /groups, GET /groups/{id}, PATCH /notifications/{id}, POST /notifications/read-all and GET /admin/reports. They follow the same token and role rules as API-01 to API-14.

Example for API-07, requesting a session:

```
POST /api/sessions
{ "requestId": 42, "tutorId": 17, "slotTime": "2026-11-02T16:00:00+05:30" }

201 Created
{ "id": 301, "status": "pending", "expiresAt": "2026-11-04T16:00:00+05:30" }
```

3.4 Error handling. All errors pass through one error-handling middleware in the API Layer, so every failure has the same JSON shape and no stack trace or database message ever reaches the user.

```
{ "status": 409, "code": "SLOT_OVERLAP", "message": "This slot overlaps an existing slot.", "requestId": "a1b2c3" }
```

HTTP status	Code	When it is returned
400	VALIDATION_ERROR	Missing, wrongly typed or too long

HTTP status	Code	When it is returned
		input; non-@pes.edu email
401	UNAUTHENTICATED, TOKEN_EXPIRED	No, wrong or expired token; wrong password
403	FORBIDDEN	User's role may not use the endpoint
403	ACCOUNT_SUSPENDED	The account was suspended by an Admin (FR-12)
404	NOT_FOUND	The record does not exist or is not visible to this user
409	CONFLICT, SLOT_OVERLAP, GROUP_FULL	Duplicate email, overlapping slot, full group, rating twice
423	ACCOUNT_LOCKED	5 failed logins in a row
429	RATE_LIMITED	More than 100 requests in a minute
500	INTERNAL_ERROR	Unexpected fault; details go to the server log only

Further rules: database changes that span several tables (for example accepting a session and creating its notification) run in one transaction and roll back on failure; a failed email never cancels a booking, because the email is retried later; each error is logged with its request ID so the team can find it; and the frontend turns each code into a plain message for the user.

3.5 Technology stack. React for the frontend, Node.js with Express for the API, PostgreSQL for the database, JSON Web Tokens for sessions, bcrypt for passwords, Jest and Supertest for tests, GitHub Actions for automation, Docker and Docker Hub for packaging, and Kubernetes for deployment.

3.6 Deployment and DevOps. Each tier runs in its own container. backend/Dockerfile builds the API image and frontend/Dockerfile builds the web image, in which nginx serves the React build, ends HTTPS (SR-01) and proxies /api/ to the API. docker-compose.yml starts the whole stack on one machine. On every push to main, GitHub Actions runs the tests, builds both images, pushes them to Docker Hub (amoghshetty05/peerlearn-api and amoghshetty05/peerlearn-web, tagged with the commit), deploys them to a Kubernetes cluster and logs in over HTTPS to confirm the deployment works (TC-20).

Kubernetes objects. The manifests in k8s/ define a peerlearn namespace; PostgreSQL as a Deployment with a 1 GiB persistent volume and a Service named db; a migration Job that applies the schema and creates the restricted database role and the first Admin; the API as a Deployment with health checks and a Service named api, kept at one replica because the background jobs must run once; and the web tier as a Deployment with two replicas behind a LoadBalancer Service on ports 80 and 443. Passwords and keys come from a Kubernetes Secret created from the local .env file, never from the repository (SO-1). k8s/deploy.sh applies these in order.